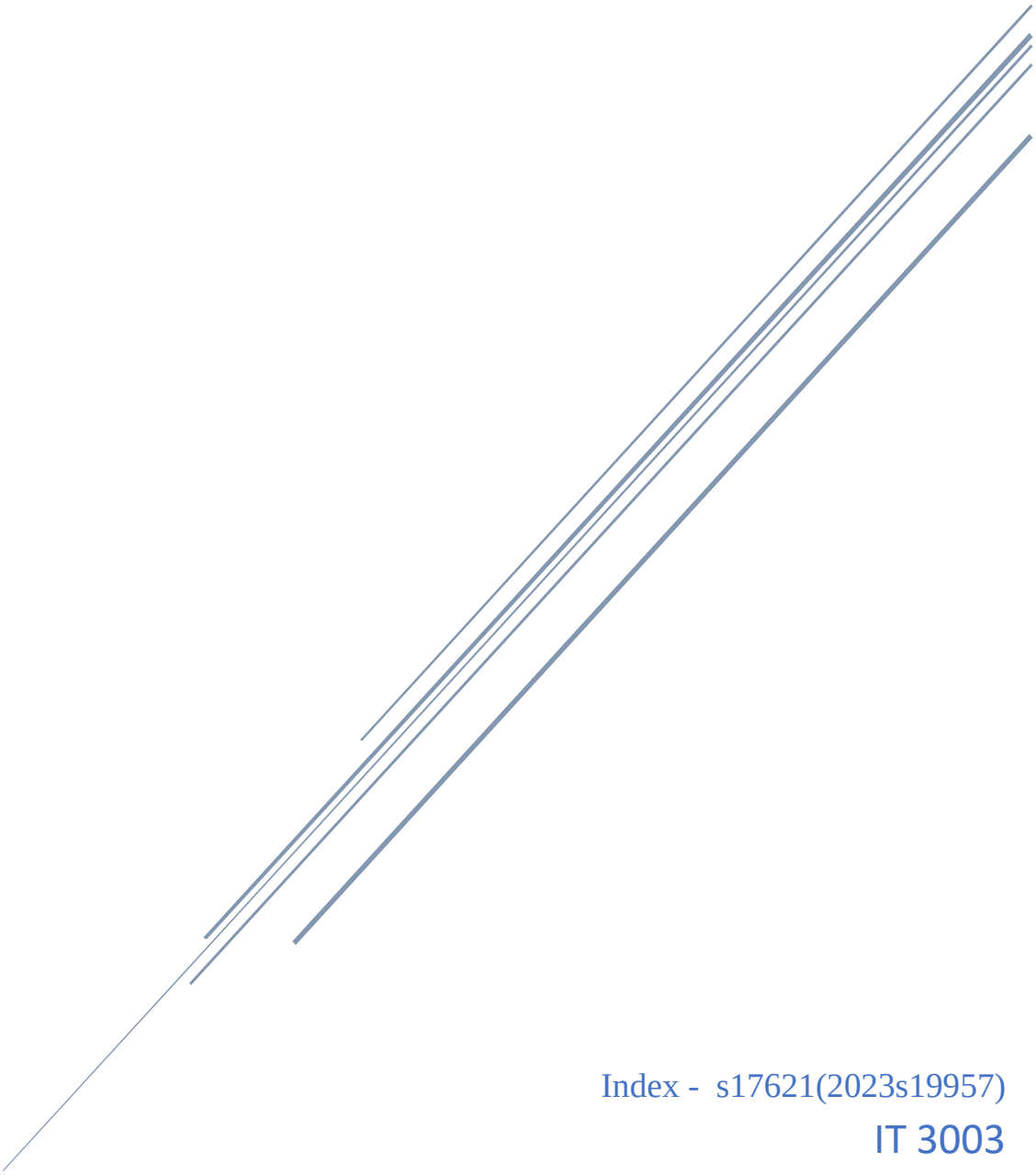
Analyzing the Role of AI-Assisted Programming in Modern Software Development

[Advanced Programming Concepts]



Index - s17621(2023s19957)

IT 3003

Table of Contents

Task 01

1.1 Introduction

Nowadays, Artificial Intelligence (AI) is becoming a part of our daily lives. We use AI in many applications such as Google Maps for finding directions, YouTube for recommending videos, and Chatgpt for answering questions. In the same way, AI is also changing the way software developers write computer programs. Instead of writing every line of code manually, developers can now use AI tools to get suggestions, generate code, find errors, and understand programming concepts more easily.

As a beginner in programming, I have experienced that writing code is not always easy. Sometimes I spend a long time trying to find a small mistake, such as a missing semicolon or a wrong variable name. There are also times when I know what I want the program to do, but I do not know how to write the code. In these situations, AI tools can provide helpful guidance and examples that make learning easier.

For example, imagine that I want to create a Student Management System in Java. If I write the program manually, I need to think about the class design, methods, loops, and conditions by myself. This helps me improve my programming skills, but it also takes more time and I may make many mistakes. On the other hand, if I use an AI tool such as Chatgpt, I can ask, "Create a Java program to add, search, update, and delete student records." The AI can generate a basic solution within a few seconds. However, I still need to read the code carefully, understand how it works, test it, and make changes if necessary. If I simply copy the code without understanding it, I will not improve my programming knowledge.

This assignment gives me the opportunity to experience both approaches. First, I will develop a software application using the traditional programming method without AI assistance. Then, I will develop the same application using an AI-assisted programming tool. After completing both versions, I will compare them based on development time, code quality, readability, maintainability, and error handling. This comparison will help me understand the strengths and weaknesses of AI-assisted programming.

I also believe that AI should be used as a learning partner rather than a replacement for programmers. Just like a calculator helps students solve mathematical calculations but does not teach mathematical thinking, AI can help programmers write code faster but cannot

replace the logical thinking and problem-solving skills that every programmer must develop. Therefore, it is important to use AI responsibly while continuing to learn the fundamentals of programming.

Through this assignment, I hope to gain a better understanding of AI-assisted programming, improve my programming skills, and learn how to use AI tools effectively and ethically in software development.

1.2 Types of AI Programming Tools

Artificial Intelligence (AI) programming tools are designed to help developers during different stages of software development. Instead of replacing programmers, these tools work as intelligent assistants that make programming easier, faster, and more efficient. Depending on the task, some AI tools can generate code, while others help find errors, improve existing code, or explain programming concepts.

As a beginner, I have learned that programming involves many activities such as writing code, debugging errors, testing programs, and documenting projects. Completing all these tasks manually can take a lot of time, especially when learning a new programming language. AI programming tools can reduce this workload by providing suggestions and guidance throughout the development process.

For example, if I want to create a Student Management System but do not know how to write a search function, I can ask an AI tool like ChatGPT to show me an example. The AI will generate sample code, explain how it works, and even suggest improvements. However, I still need to understand the code instead of simply copying it.

There are several types of AI programming tools, and each type has a different purpose. The following sections describe the most common types used in modern software development.

AI-assisted programming tools can be classified into several categories depending on their primary function.

1.2.1 Code Generation Tools

Code generation tools automatically create source code based on natural language descriptions or partially written code. Developers describe the required functionality, and the AI generates complete methods, classes, or even entire programs.

Examples include:

- ChatGPT
- GitHub Copilot
- Amazon Q Developer
- Google Gemini Code Assist

These tools significantly reduce development time and help developers implement features more quickly.

1.2.2 Code Completion Tools

Code completion tools predict the next word, statement, or block of code while the programmer is typing. They help reduce typing effort and minimize syntax errors.

Features include:

- Auto-completion
- Function suggestions
- Variable name prediction
- Parameter suggestions

These tools improve programming speed and reduce repetitive coding tasks.

1.2.3 Debugging Assistants

Debugging assistants help programmers identify and fix errors in software. They can analyze compiler messages, runtime exceptions, and logical errors, then suggest possible solutions.

Capabilities include:

- Finding syntax errors
- Explaining runtime exceptions
- Suggesting bug fixes
- Improving program stability

These tools reduce debugging time and make problem-solving easier for beginners.

1.2.4 Refactoring Tools

Refactoring tools improve the structure and readability of existing code without changing its functionality.

Common refactoring tasks include:

- Renaming variables and methods
- Removing duplicate code
- Simplifying complex functions
- Improving code organization

Refactoring tools make software easier to maintain and extend.

1.2.5 Documentation and Explanation Tools

Some AI tools generate documentation automatically or explain how existing code works.

Examples include:

- Generating JavaDoc comments
- Explaining algorithms
- Creating README files
- Producing code summaries

These tools improve communication among software developers and make projects easier to understand.

1.3 Advantages of AI-Assisted Programming

Artificial Intelligence (AI) has changed the way software is developed. Today, many developers use AI tools to help them write, test, debug, and improve their programs. These tools make programming easier by providing suggestions, generating code, and explaining difficult concepts. As a beginner, I have found that AI-assisted programming is not only useful for completing programming tasks but also for learning new programming techniques. Although AI cannot replace human programmers, it can make software development faster and more efficient.

The following are some of the major advantages of AI-assisted programming.

- Faster Development

AI can generate code quickly, reducing the time required to develop software. Developers spend less time writing repetitive code and more time solving complex problems.

- Increased Productivity

Programmers can complete more work in less time because AI automates repetitive tasks such as creating loops, methods, classes, and documentation.

- Better Code Quality

Many AI tools recommend coding best practices and help developers write cleaner, more organized, and more maintainable code.

- Reduced Programming Errors

AI can identify syntax mistakes, missing brackets, incorrect variable names, and other common programming errors before the program is executed.

- Learning Support

Students and beginner programmers can use AI as a learning assistant. AI explains programming concepts, algorithms, object-oriented programming, and difficult code examples in simple language.

- Improved Documentation

AI can automatically generate comments, documentation, and project descriptions, reducing documentation effort.

- Better Problem Solving

AI suggests alternative algorithms, data structures, and optimization techniques that developers may not have considered.

1.4 Risks and Limitations of AI-Assisted Programming

Although AI-assisted programming provides many benefits, it also has several risks and limitations. AI tools can help developers write code faster and solve problems more easily, but they are not perfect. Sometimes AI generates incorrect code, misunderstands the user's requirements, or provides outdated information. Therefore, developers should use AI carefully and always verify the generated code before using it in real applications.

As a beginner, I have noticed that AI can quickly provide answers, but not every answer is correct. If I copy the code without understanding it, I may face problems later when debugging or modifying the program. For this reason, AI should be used as a learning assistant rather than a complete replacement for programming knowledge.

The following are some of the main risks and limitations of AI-assisted programming.

- Hallucinated Code

AI sometimes generates incorrect or non-functional code that appears correct. Developers must carefully test all generated programs before using them.

- Security Risks

AI-generated code may contain security vulnerabilities such as SQL injection, weak authentication mechanisms, or insecure data handling practices if not reviewed carefully.

- Over-Dependence

Students may become overly dependent on AI and fail to develop essential programming skills. This can negatively affect their ability to solve programming problems independently.

- Incorrect Logic

AI may misunderstand the programmer's requirements and produce code that compiles successfully but does not solve the intended problem.

- Outdated Information

Some AI tools may generate code based on outdated programming libraries or older software versions, requiring developers to update the generated code.

- Privacy Concerns

When developers submit confidential source code to online AI services, sensitive information may be exposed if appropriate security measures are not followed.

- Copyright and Licensing Issues

AI-generated code may resemble existing open-source software. Developers should verify licensing requirements and ensure compliance with intellectual property laws.

In this task, I learned that **AI-assisted programming** is the use of Artificial Intelligence tools to help developers write, debug, improve, and understand code more efficiently. There are different types of AI programming tools, such as code generation, code completion, debugging, refactoring, and documentation tools, each serving a different purpose in software development.

I also learned that AI-assisted programming offers many advantages, including faster software development, improved productivity, better code quality, and valuable learning support for beginners. However, it also has some risks, such as generating incorrect code, creating security concerns, and encouraging over-dependence on AI if users rely on it without understanding the code.

Overall, AI is a powerful tool that can make programming easier and more efficient, but it should be used responsibly. Developers should always understand, test, and verify AI-generated code while continuing to build their own programming knowledge and problem-solving skills.

Task 02

2.1 Select Program

I select Student Management System project

Many educational institutions manage a large number of student records. Managing these records manually can be time-consuming and may lead to errors such as duplicate entries or missing information. Therefore, a Student Management System is developed to store and manage student information efficiently.

The system allows users to:

- Add a new student
- Search for a student
- Update student details
- Delete student records
- Display all students

This project is implemented using Java and follows Object-Oriented Programming (OOP) principles. Two versions of the system are developed: one using the traditional programming approach and another using AI-assisted programming. The two implementations are compared to evaluate the impact of AI on software development.

2.2 Student Management System Code

Version A – Basic java Function and OOP concept without AI

Code:

```
import java.util.ArrayList;
import java.util.Scanner;

public class StudentManagementSystem {

    // Holds all student records in memory
    private static ArrayList<Student> students = new ArrayList<>();
    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int choice;
```

```
        do {
            printMenu();
            choice = readInt("Enter your choice: ");

            switch (choice) {
                case 1:
                    addStudent();
                    break;
                case 2:
                    searchStudent();
                    break;
                case 3:
                    updateStudent();
                    break;
                case 4:
                    deleteStudent();
                    break;
                case 5:
                    displayStudents();
                    break;
                case 6:
                    System.out.println("Exiting... Goodbye!");
                    break;
                default:
                    System.out.println("Invalid choice. Please select a number between 1 and 6.");
            }

            System.out.println();
        } while (choice != 6);

        scanner.close();
    }

    private static void printMenu() {
        System.out.println("===== Student Management System =====");
        System.out.println("1. Add Student");
        System.out.println("2. Search Student");
        System.out.println("3. Update Student");
        System.out.println("4. Delete Student");
        System.out.println("5. Display Students");
        System.out.println("6. Exit");
    }

    // ---------- Core operations ----------

    private static void addStudent() {
        System.out.println("--- Add Student ---");

        int id = readInt("Enter ID: ");

        // Prevent duplicate IDs
        if (findStudentById(id) != null) {
            System.out.println("A student with ID " + id + " already exists.");
            return;
        }
```

```
    String name = readString("Enter Name: ");
    int age = readInt("Enter Age: ");
    String course = readString("Enter Course: ");

    students.add(new Student(id, name, age, course));
    System.out.println("Student added successfully.");
}

private static void searchStudent() {
    System.out.println("--- Search Student ---");
    int id = readInt("Enter ID to search: ");

    Student found = findStudentById(id);
    if (found != null) {
        System.out.println("Student found:");
        System.out.println(found);
    } else {
        System.out.println("No student found with ID " + id);
    }
}

private static void updateStudent() {
    System.out.println("--- Update Student ---");
    int id = readInt("Enter ID of student to update: ");

    Student student = findStudentById(id);
    if (student == null) {
        System.out.println("No student found with ID " + id);
        return;
    }

    System.out.println("Current details: " + student);
    System.out.println("Leave a field blank to keep its current value.");

    String name = readString("Enter new Name (" + student.getName() + "): ");
    if (!name.isEmpty()) {
        student.setName(name);
    }

    String ageInput = readString("Enter new Age (" + student.getAge() + "): ");
    if (!ageInput.isEmpty()) {
        try {
            student.setAge(Integer.parseInt(ageInput));
        } catch (NumberFormatException e) {
            System.out.println("Invalid age entered. Age not updated.");
        }
    }

    String course = readString("Enter new Course (" + student.getCourse() + "): ");
    if (!course.isEmpty()) {
        student.setCourse(course);
    }

    System.out.println("Student updated successfully.");
```

```
    }

    private static void deleteStudent() {
        System.out.println("--- Delete Student ---");
        int id = readInt("Enter ID of student to delete: ");

        Student student = findStudentById(id);
        if (student == null) {
            System.out.println("No student found with ID " + id);
            return;
        }

        students.remove(student);
        System.out.println("Student deleted successfully.");
    }

    private static void displayStudents() {
        System.out.println("--- All Students ---");

        if (students.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }

        for (Student s : students) {
            System.out.println(s);
        }
    }

    // ---------- Helper methods ----------

    private static Student findStudentById(int id) {
        for (Student s : students) {
            if (s.getId() == id) {
                return s;
            }
        }
        return null;
    }

    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = scanner.nextLine().trim();
            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid number.");
            }
        }
    }

    private static String readString(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
```

```
    }
}

// Student is kept in the same file as a non-public class.
// Java allows this as long as only one class in the file is public,
// and the public class's name matches the file name.
class Student {
    private int id;
    private String name;
    private int age;
    private String course;

    public Student(int id, String name, int age, String course) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.course = course;
    }

    // Getters
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getCourse() {
        return course;
    }

    // Setters
    public void setId(int id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
```

```
        return "ID: " + id +
                " | Name: " + name +
                " | Age: " + age +
                " | Course: " + course;
        }
}
```

Version B – Using AI (add Error hadling,use advance concept)

Code:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.Scanner;
import java.util.function.Consumer;
import java.util.logging.Level;
import java.util.logging.Logger;
import java.util.stream.Collectors;

/**
 * Entry point and console UI for the Student Management System.
 * <p>
 * Responsibilities are deliberately separated:
 * - {@link Student}                -> data + field-level validation
 * - {@link StudentRepository}      -> storage contract (CRUD)
 * - {@link InMemoryStudentRepository} -> in-memory implementation backed by a Map
 * - {@link StudentManagementSystemAICODE} -> menu, input handling, orchestration
 * <p>
 * This keeps business rules (what a valid student looks like, how storage works)
 * independent of how the user interacts with the program, so either side can be
 * swapped or extended (e.g. a database-backed repository, or a GUI front end)
 * without touching the other.
 */
public class StudentManagementSystemAICODE {

    private static final Logger LOGGER =
Logger.getLogger(StudentManagementSystemAICODE.class.getName());
    private static final Scanner SCANNER = new Scanner(System.in);
    private static final StudentRepository repository = new InMemoryStudentRepository();

    public static void main(String[] args) {
        runApplication();
    }

    /** Main program loop: show menu, read a choice, dispatch, repeat until Exit. */
    private static void runApplication() {
        boolean running = true;

        while (running) {
            printMenu();
            MenuOption option = readMenuOption();

            try {
                switch (option) {
                    case ADD -> addStudent();
                    case SEARCH -> searchStudent();
                    case UPDATE -> updateStudent();
```

```
                case DELETE -> deleteStudent();
                case DISPLAY -> displayStudents();
                case EXIT -> running = false;
            }
        } catch (StudentNotFoundException | DuplicateStudentException | InvalidInputException e) {
            // Expected, user-facing failures: show a clean message, keep the app running.
            System.out.println("Error: " + e.getMessage());
        } catch (Exception e) {
            // Anything unanticipated: log the full detail for diagnosis, but never crash the UI.
            LOGGER.log(Level.SEVERE, "Unexpected error occurred", e);
            System.out.println("An unexpected error occurred. Please try again.");
        }

        System.out.println();
    }

    System.out.println("Goodbye!");
    SCANNER.close();
}

// ---------- Menu ----------

private static void printMenu() {
    System.out.println("===== Student Management System =====");
    for (MenuOption option : MenuOption.values()) {
        System.out.println(option.getCode() + ". " + option.getLabel());
    }
}

// ---------- Feature operations ----------

private static void addStudent() throws InvalidInputException {
    System.out.println("--- Add Student ---");

    int id = readPositiveInt("Enter ID: ");
    if (repository.existsById(id)) {
        System.out.println("A student with ID " + id + " already exists.");
        return;
    }

    String name = readNonEmptyString("Enter Name: ");
    int age = readIntInRange("Enter Age: ", 1, 120);
    String course = readNonEmptyString("Enter Course: ");

    try {
        Student student = new Student(id, name, age, course);
        repository.add(student);
        System.out.println("Student added successfully.");
    } catch (IllegalArgumentException e) {
        // Student's own validation caught something the input helpers didn't.
        throw new InvalidInputException(e.getMessage());
    }
}

private static void searchStudent() throws InvalidInputException {
```

```
    System.out.println("--- Search Student ---");
    System.out.println("1. Search by ID");
    System.out.println("2. Search by Name");
    int choice = readIntInRange("Choose an option: ", 1, 2);

    if (choice == 1) {
        int id = readPositiveInt("Enter ID: ");
        repository.findById(id).ifPresentOrElse(
                s -> System.out.println("Student found:\n" + s),
                () -> System.out.println("No student found with ID " + id)
        );
    } else {
        String keyword = readNonEmptyString("Enter name keyword: ");
        List<Student> matches = repository.findByNameContains(keyword);

        if (matches.isEmpty()) {
            System.out.println("No students matched \"" + keyword + "\".");
        } else {
            printTableHeader();
            matches.forEach(System.out::println);
        }
    }
}

private static void updateStudent() throws InvalidInputException {
    System.out.println("--- Update Student ---");
    int id = readPositiveInt("Enter ID of student to update: ");

    Student existing = repository.findById(id)
            .orElseThrow(() -> new StudentNotFoundException("No student found with ID " + id));

    System.out.println("Current details:\n" + existing);
    System.out.println("Leave a field blank to keep its current value.");

    String name = readOptionalString("New Name (" + existing.getName() + "): ");
    String ageInput = readOptionalString("New Age (" + existing.getAge() + "): ");
    String course = readOptionalString("New Course (" + existing.getCourse() + "): ");

    try {
        repository.update(id, student -> {
            if (!name.isEmpty()) {
                student.setName(name);
            }
            if (!ageInput.isEmpty()) {
                student.setAge(Integer.parseInt(ageInput));
            }
            if (!course.isEmpty()) {
                student.setCourse(course);
            }
        });
        System.out.println("Student updated successfully.");
    } catch (NumberFormatException e) {
        throw new InvalidInputException("Age must be a whole number.");
    } catch (IllegalArgumentException e) {
        throw new InvalidInputException(e.getMessage());
```

```
        }
    }

    private static void deleteStudent() {
        System.out.println("--- Delete Student ---");
        int id = readPositiveInt("Enter ID of student to delete: ");
        repository.delete(id); // throws StudentNotFoundException, handled by the caller's catch block
        System.out.println("Student deleted successfully.");
    }

    private static void displayStudents() {
        System.out.println("--- All Students ---");
        List<Student> all = repository.findAll();

        if (all.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }

        System.out.println("Sort by: 1. ID   2. Name   3. Age   (press Enter for ID)");
        String sortChoice = readOptionalString("Choice: ");

        Comparator<Student> comparator = switch (sortChoice) {
            case "2" -> Comparator.comparing(Student::getName, String.CASE_INSENSITIVE_ORDER);
            case "3" -> Comparator.comparingInt(Student::getAge);
            default -> Comparator.comparingInt(Student::getId);
        };

        printTableHeader();
        all.stream().sorted(comparator).forEach(System.out::println);
    }

    private static void printTableHeader() {
        System.out.printf("%-6s %-20s %-5s %-15s%n", "ID", "Name", "Age", "Course");
        System.out.println("-".repeat(50));
    }

    // ---------- Input helpers (all retry-on-invalid, never throw for bad input) ----------

    private static int readPositiveInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = SCANNER.nextLine().trim();
            try {
                int value = Integer.parseInt(input);
                if (value <= 0) {
                    System.out.println("Please enter a positive number.");
                    continue;
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid whole number.");
            }
        }
    }
```

```
    private static int readIntInRange(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            String input = SCANNER.nextLine().trim();
            try {
                int value = Integer.parseInt(input);
                if (value < min || value > max) {
                    System.out.println("Please enter a number between " + min + " and " + max + ".");
                    continue;
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid whole number.");
            }
        }
    }

    private static String readNonEmptyString(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = SCANNER.nextLine().trim();
            if (!input.isEmpty()) {
                return input;
            }
            System.out.println("This field cannot be empty.");
        }
    }

    /** Like readNonEmptyString, but an empty answer is valid (used by "leave blank to skip"
prompts). */
    private static String readOptionalString(String prompt) {
        System.out.print(prompt);
        return SCANNER.nextLine().trim();
    }

    private static MenuOption readMenuOption() {
        while (true) {
            System.out.print("Enter your choice: ");
            String input = SCANNER.nextLine().trim();
            try {
                int code = Integer.parseInt(input);
                return MenuOption.fromCode(code);
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid number.");
            } catch (InvalidInputException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}

//
============================================================================
=============
```

```
// Domain model
//
================================================================================
=============

/**
 * Represents a single student record.
 * Validation lives here (not in the UI layer) so a Student object is never in an
 * invalid state, regardless of who constructs or edits it.
 */
class Student implements Comparable<Student> {
    private int id;
    private String name;
    private int age;
    private String course;

    public Student(int id, String name, int age, String course) {
        setId(id);
        setName(name);
        setAge(age);
        setCourse(course);
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getCourse() {
        return course;
    }

    public void setId(int id) {
        if (id <= 0) {
            throw new IllegalArgumentException("ID must be a positive integer.");
        }
        this.id = id;
    }

    public void setName(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name cannot be empty.");
        }
        this.name = name.trim();
    }

    public void setAge(int age) {
        if (age < 1 || age > 120) {
```

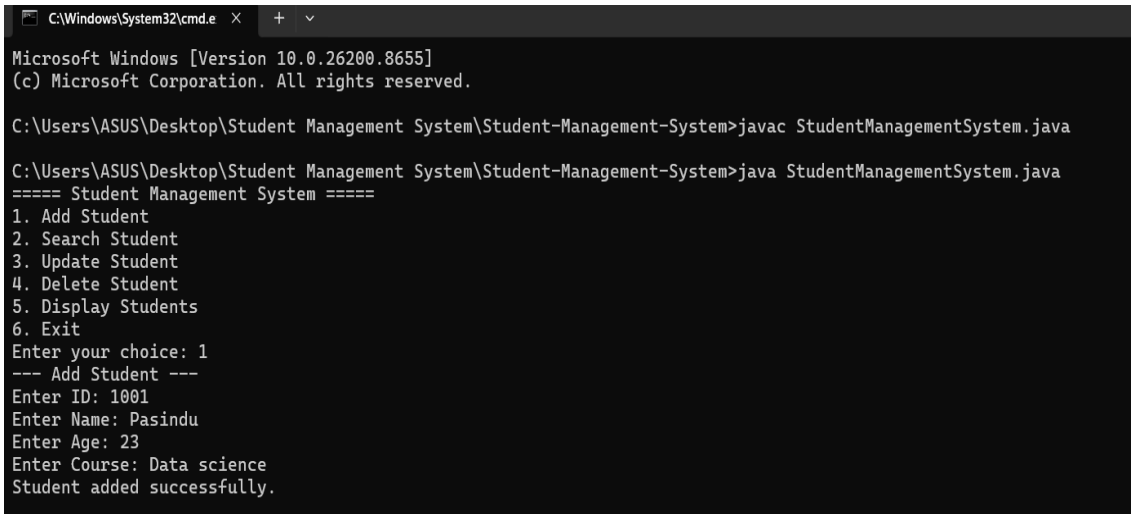
```
            throw new IllegalArgumentException("Age must be between 1 and 120.");
        }
        this.age = age;
    }

    public void setCourse(String course) {
        if (course == null || course.isBlank()) {
            throw new IllegalArgumentException("Course cannot be empty.");
        }
        this.course = course.trim();
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.id, other.id);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Student)) {
            return false;
        }
        Student other = (Student) o;
        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }

    @Override
    public String toString() {
        return String.format("%-6d %-20s %-5d %-15s", id, name, age, course);
    }
}

/** The six actions the console menu supports, each tied to its displayed number and label. */
enum MenuOption {
    ADD(1, "Add Student"),
    SEARCH(2, "Search Student"),
    UPDATE(3, "Update Student"),
    DELETE(4, "Delete Student"),
    DISPLAY(5, "Display Students"),
    EXIT(6, "Exit");

    private final int code;
    private final String label;

    MenuOption(int code, String label) {
        this.code = code;
        this.label = label;
```

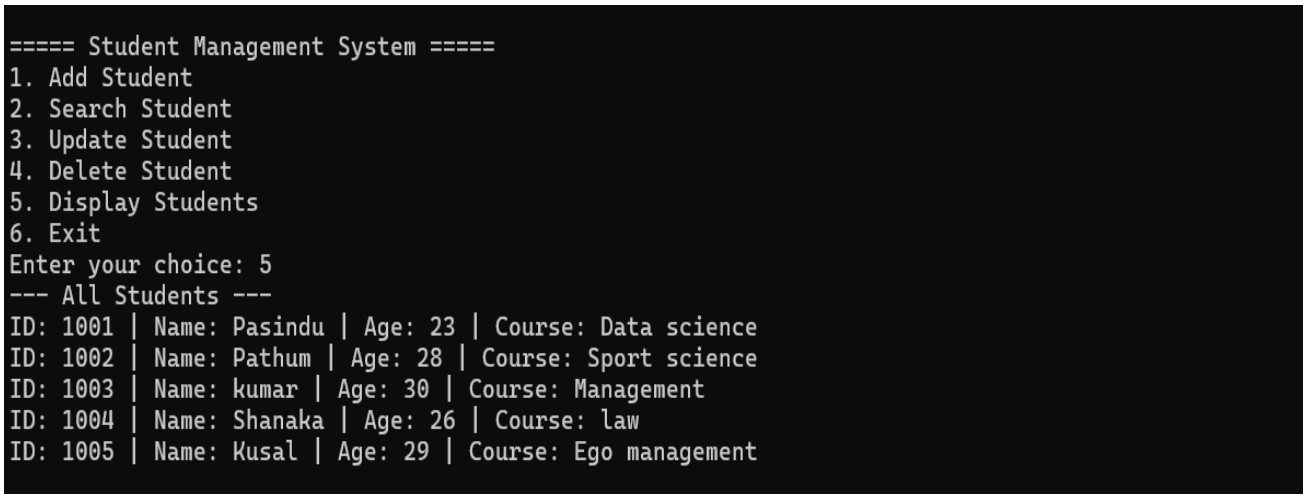
```
    }

    public int getCode() {
        return code;
    }

    public String getLabel() {
        return label;
    }

    public static MenuOption fromCode(int code) throws InvalidInputException {
        for (MenuOption option : values()) {
            if (option.code == code) {
                return option;
            }
        }
        throw new InvalidInputException("Invalid menu choice: " + code + ". Pick a number from 1 to
6.");
    }
}

//
==========================================================================
=============
// Persistence layer (in-memory today, swappable for a database later)
//
==========================================================================
=============

/** Storage contract for students, independent of how or where records are kept. */
interface StudentRepository {
    void add(Student student) throws DuplicateStudentException;

    Optional<Student> findById(int id);

    List<Student> findByNameContains(String keyword);

    void update(int id, Consumer<Student> updater) throws StudentNotFoundException;

    void delete(int id) throws StudentNotFoundException;

    List<Student> findAll();

    boolean existsById(int id);
}

/** Keeps students in a Map keyed by ID for O(1) lookup, update, and delete. */
class InMemoryStudentRepository implements StudentRepository {
    // LinkedHashMap preserves insertion order, so "Display Students" shows a stable order by default.
    private final Map<Integer, Student> studentsById = new LinkedHashMap<>();

    @Override
    public void add(Student student) throws DuplicateStudentException {
        if (studentsById.containsKey(student.getId())) {
            throw new DuplicateStudentException("Student with ID " + student.getId() + " already
```

```
exists.");
        }
        studentsById.put(student.getId(), student);
    }

    @Override
    public Optional<Student> findById(int id) {
        return Optional.ofNullable(studentsById.get(id));
    }

    @Override
    public List<Student> findByNameContains(String keyword) {
        String lowerKeyword = keyword.toLowerCase();
        return studentsById.values().stream()
                .filter(s -> s.getName().toLowerCase().contains(lowerKeyword))
                .collect(Collectors.toList());
    }

    @Override
    public void update(int id, Consumer<Student> updater) throws StudentNotFoundException {
        Student student = studentsById.get(id);
        if (student == null) {
            throw new StudentNotFoundException("No student found with ID " + id);
        }
        updater.accept(student);
    }

    @Override
    public void delete(int id) throws StudentNotFoundException {
        if (studentsById.remove(id) == null) {
            throw new StudentNotFoundException("No student found with ID " + id);
        }
    }

    @Override
    public List<Student> findAll() {
        return new ArrayList<>(studentsById.values());
    }

    @Override
    public boolean existsById(int id) {
        return studentsById.containsKey(id);
    }
}

//
============================================================================
=============
// Custom exceptions -- each names exactly one failure mode, so callers can react precisely
//
============================================================================
=============

/** Thrown when an operation targets a student ID that isn't in the repository. */
class StudentNotFoundException extends RuntimeException {
```

```
    public StudentNotFoundException(String message) {
        super(message);
    }
}

/** Thrown when trying to add a student whose ID already exists. */
class DuplicateStudentException extends RuntimeException {
    public DuplicateStudentException(String message) {
        super(message);
    }
}

/** Thrown for user-supplied data that fails validation (checked, so callers must handle it deliberately). */
class InvalidInputException extends Exception {
    public InvalidInputException(String message) {
        super(message);
    }
}
```

2.3 OUTPUTS – Screenshorts





```
===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 5
--- All Students ---
ID: 1001 | Name: Pasindu | Age: 23 | Course: Data science
ID: 1002 | Name: Pathum | Age: 28 | Course: Sport science
ID: 1003 | Name: kumar | Age: 30 | Course: Management
ID: 1004 | Name: Dasun | Age: 27 | Course: Cricket
ID: 1005 | Name: Kusal | Age: 29 | Course: Ego management

===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 4
--- Delete Student ---
Enter ID of student to delete: 1005
Student deleted successfully.
```

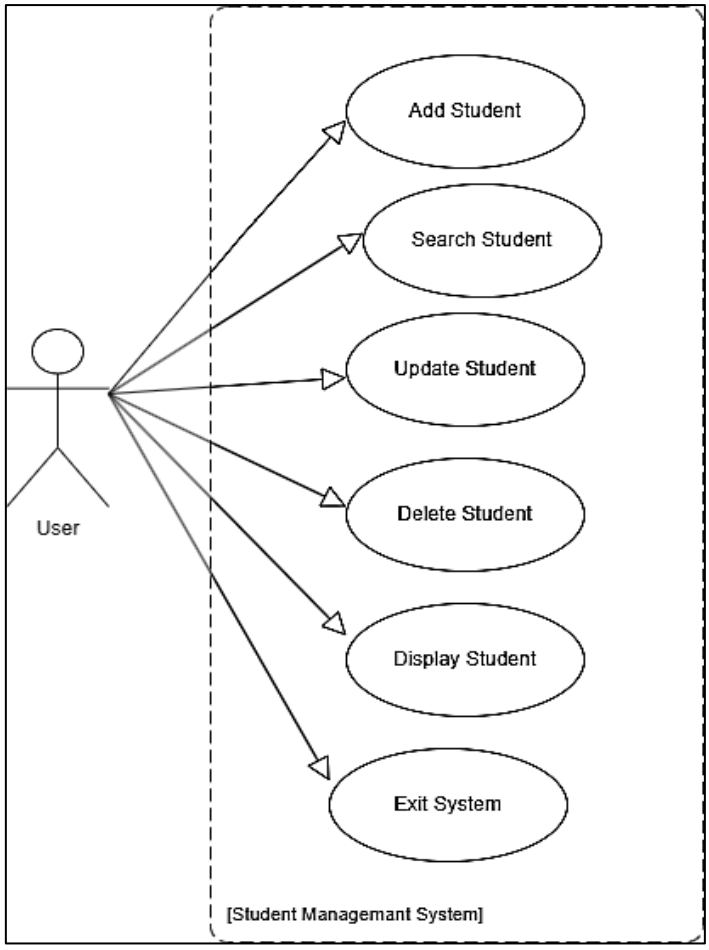
```
===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 5
--- All Students ---
ID: 1001 | Name: Pasindu | Age: 23 | Course: Data science
ID: 1002 | Name: Pathum | Age: 28 | Course: Sport science
ID: 1003 | Name: kumar | Age: 30 | Course: Management
ID: 1004 | Name: Dasun | Age: 27 | Course: Cricket
```

```
===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 3
--- Update Student ---
Enter ID of student to update: 1004
Current details: ID: 1004 | Name: Shanaka | Age: 26 | Course: law
Leave a field blank to keep its current value.
Enter new Name (Shanaka): Dasun
Enter new Age (26): 27
Enter new Course (law): Cricket
Student updated successfully.
```

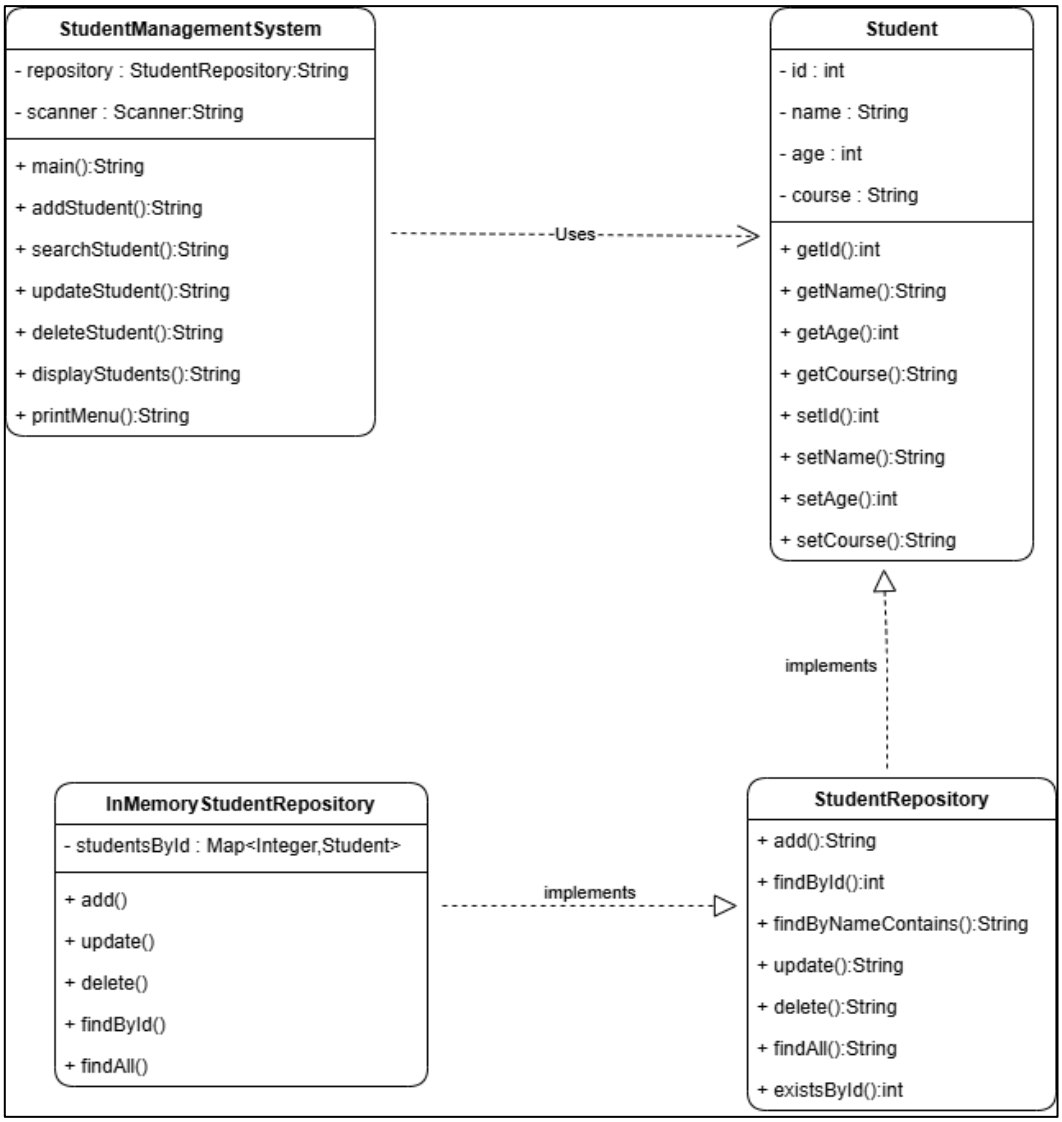
```
===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 5
--- All Students ---
ID: 1001 | Name: Pasindu | Age: 23 | Course: Data science
ID: 1002 | Name: Pathum | Age: 28 | Course: Sport science
ID: 1003 | Name: kumar | Age: 30 | Course: Management
ID: 1004 | Name: Dasun | Age: 27 | Course: Cricket

===== Student Management System =====
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display Students
6. Exit
Enter your choice: 6
Exiting... Goodbye!
```

2.4 UML Diagram

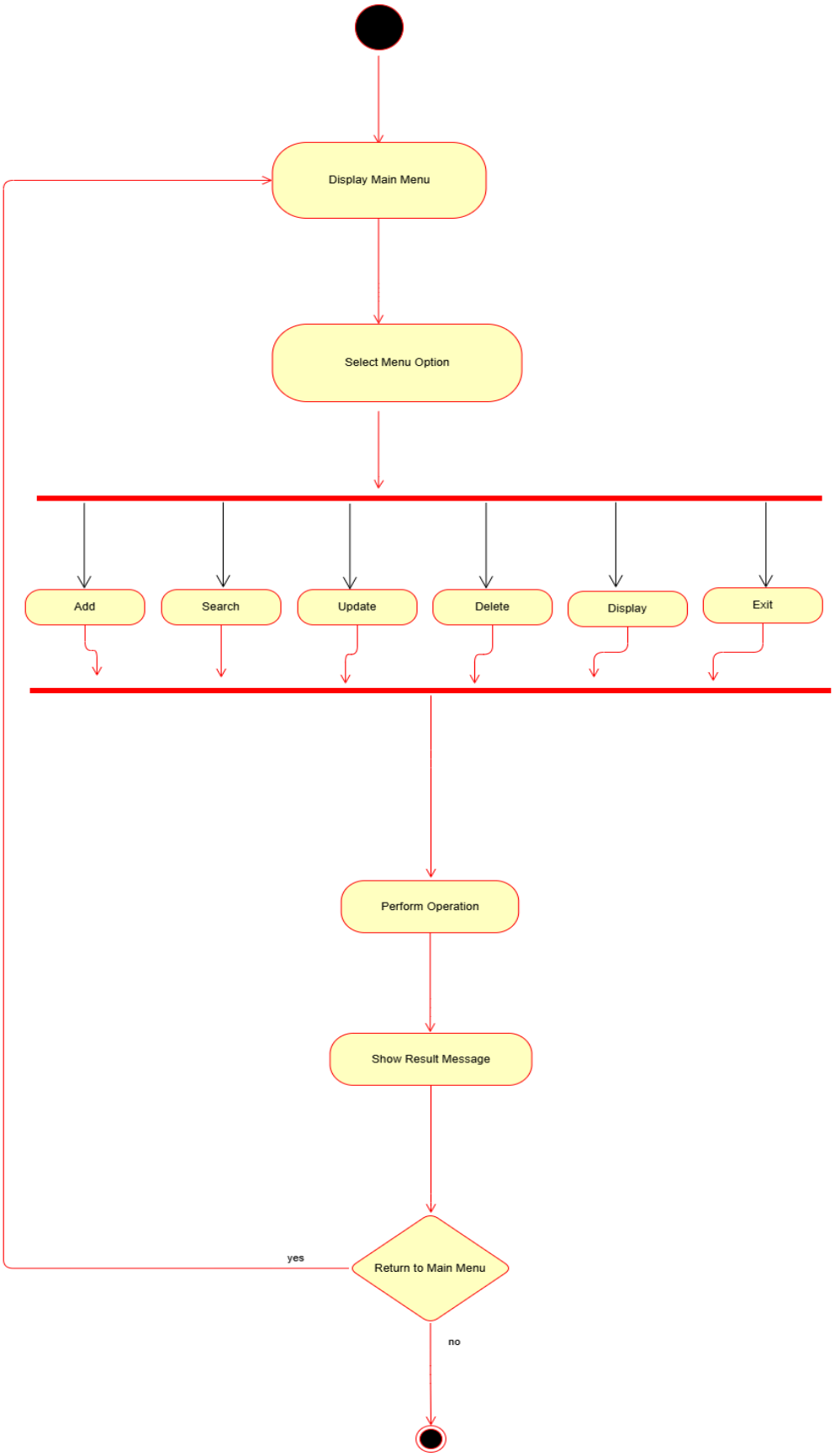
1) Use Case Diagram



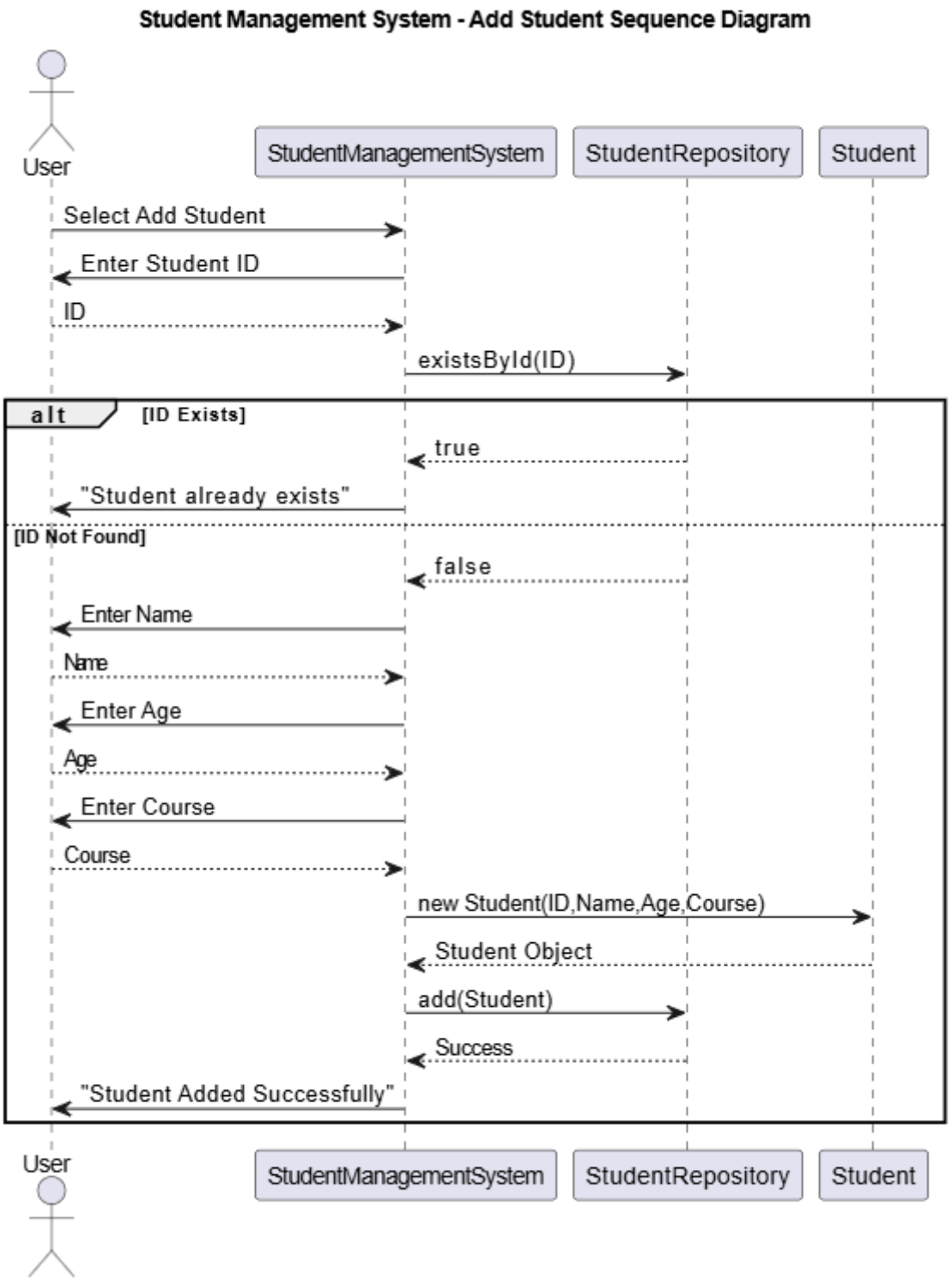
2) Class Diagram



3) Activity Diagram



4) Sequence Diagram



2.5 Student Management System Comparison

Basic Version vs. Advanced Version

This report compares two versions of the same Student Management System developed in Java. Both versions perform the same basic functions, such as adding, searching, updating, deleting, and displaying student records. However, they are developed using different programming approaches.

The Basic Version was created using simple Object-Oriented Programming (OOP) concepts. It is easy for beginners to understand.

The Advanced Version includes additional Java features such as interfaces, custom exceptions, repository design, and improved error handling. Although it is more complex, it is better organized and easier to expand in the future.

The purpose of this comparison is to understand the differences between a simple Java program and a more advanced implementation.

While developing both versions, I noticed several differences.

The Basic Version was much easier to write because it uses only basic Java concepts such as classes, objects, methods, loops, and Array list. As a beginner, I found it easier to understand how the program works because all the code is straightforward.

The Advanced Version introduces new concepts such as interfaces, custom exceptions, repository patterns, and Java Collections. At first, these concepts were difficult to understand. However, after studying the code, I realized that separating the program into different classes makes it more organized and easier to manage.

Another important difference is error handling. In the basic program, only simple input errors are handled. In the advanced version, different types of errors are managed separately, making the application more reliable.

The advanced version also performs better when searching for student records because it uses a Map instead of an Array list. This makes searching faster, especially when the number of students increases.

Comparision:

Feature	Basic Version	Advanced Version
Program Structure	Uses only two classes, making it simple and easy to understand.	Uses several classes and interfaces to organize the program into separate parts.
Data Storage	Stores student records using an `ArrayList`.	Stores student records using a `Map`, making searches faster.
Input Validation	Only basic validation is used. Invalid data may still be accepted.	Validates data before storing it and prevents invalid student records.
Error Handling	Uses simple `try-catch` blocks for input errors.	Uses custom exceptions to handle different errors more effectively.
Menu System	Uses simple switch-case statements with numbers.	Uses an Enum to make the menu easier to manage.
Search Function	Searches students only by ID.	Can search by both student ID and student name.
Display Options	Displays students in the order they were added.	Can sort students by ID, name, or age before displaying them.
Future Improvements	Adding new features requires changing the main program.	New features can be added more easily because the program is well organized.
Code Organization	Simple but some code is repeated.	Common tasks are reused, reducing duplicate code.
Readability	Easy for beginners to understand.	More structured but slightly harder for beginners.
Maintainability	Difficult to maintain when the project becomes larger.	Easier to maintain because each class has a separate responsibility.
Testing	Testing is difficult because all logic is inside the main program.	Easier to test because each class works independently.

Advantages of the Basic Version	Advantages of the Advanced Version
□ Easy to understand for beginners. □ Uses simple Java concepts. □ Less code to read. □ Easy to explain during presentations. □ Suitable for small projects	□ Better program organization. □ Improved error handling. □ Faster searching using Map. □ Easier to add new features. □ Better code quality and maintainability. □ More suitable for large software projects.

Both versions successfully perform the required functions of the Student Management System. The Basic Version is suitable for beginners because it is simple, easy to understand, and demonstrates the fundamental concepts of Java programming.

The Advanced Version is more suitable for real-world software development because it provides better organization, stronger error handling, improved maintainability, and higher performance. Although it is more difficult to understand at first, it follows good software engineering practices.

Overall, this comparison helped me understand the importance of writing not only functional code but also clean and maintainable code. As I continue learning Java, I hope to gradually apply these advanced programming techniques in future software projects.

2.6 AI Prompt

1- AI Code

Create an advanced console-based Student Management System in Java using Object-Oriented Programming (OOP).

Features:

1. Add Student

2. Search Student (by ID and Name)

3. Update Student

4. Delete Student

5. Display Students (sort by ID, Name, or Age)

6. Exit

Requirements:

- Use multiple classes (Student, StudentRepository interface, InMemoryStudentRepository, StudentManagementSystem, MenuOption enum).

- Store students using Map<Integer, Student>.

- Implement CRUD operations.

- Use interfaces, encapsulation, polymorphism, enums, Optional, Stream API, Comparator, and method references.

- Create custom exceptions (DuplicateStudentException, StudentNotFoundException, InvalidInputException).

- Validate all user input and student data.

- Handle errors gracefully without crashing.

- Use clean, well-commented, beginner-friendly code.

Output:

- Complete Java source code for each file.

- Project folder structure.

- Brief explanation of the OOP concepts and advanced Java features used.

Task 3

Critical Analysis

3.1 How AI Changed My Coding Approach

Before using AI, I usually started writing my programs from the beginning by thinking about the program structure and writing each method one by one. When I faced an error, I searched Google or watched YouTube videos to find a solution. This process took a lot of time.

After using AI tools such as ChatGPT, my coding approach changed. Instead of searching different websites, I could ask AI questions directly and receive code examples and explanations within a few seconds. AI also suggested better ways to organize my code and helped me understand programming concepts that I found difficult. However, I realized that I should not copy the code directly. I needed to read it carefully, understand how it works, and make changes according to my project requirements.

3.2 Did AI Improve or Reduce My Understanding?

In my experience, AI improved my understanding when I used it correctly. Whenever I did not understand a Java concept or an error message, AI explained it in simple language with examples. This helped me learn faster and complete my project more confidently.

However, I also realized that if I simply copy and paste AI-generated code without reading it, I will not improve my programming skills. Therefore, AI is most useful when it is used as a learning tool rather than a shortcut for completing assignments.

Overall, I believe AI improved my understanding because it allowed me to learn from examples and explanations while still encouraging me to practice programming.

3.3 When Should AI NOT Be Used?

Although AI is very helpful, there are situations where it should not be used.

First, AI should not be used during examinations or assessments where students are expected to complete their own work without assistance.

Second, AI should not be trusted completely when developing important software such as banking systems, hospital management systems, or security applications. In these situations, every line of code should be carefully reviewed and tested because mistakes could have serious consequences.

Finally, AI should not replace independent thinking. Students should first try to solve programming problems by themselves before asking AI for help. This helps improve logical thinking and problem-solving skills.

3.4 Ethical Considerations in AI-Generated Code

Using AI in programming also involves ethical responsibilities. Developers should use AI honestly and responsibly. AI-generated code should always be checked to make sure it is correct, secure, and suitable for the project.

Students should not submit AI-generated code as their own work without understanding it, especially if their university has rules about the use of AI tools. It is also important to avoid using AI to create harmful or illegal software.

In my opinion, AI should be treated as a learning assistant rather than a replacement for programmers. By using AI responsibly, students can improve their knowledge while maintaining academic honesty and professional ethics.

Task 4

Conclusion

Final Opinion on AI in Programming Education and Industry

In my opinion, Artificial Intelligence is a valuable tool for both programming education and the software industry. For students, AI makes learning easier by providing explanations, examples, and support when solving programming problems. It helps beginners understand difficult concepts and reduces the time spent searching for solutions. However, students should not depend on AI for every task. It is important to understand the code, practice regularly, and develop problem-solving skills on their own.

In the software industry, AI helps developers work more efficiently by generating code, finding errors, improving code quality, and automating repetitive tasks. This allows developers to focus on solving complex problems and creating better software. However, AI-generated code should always be reviewed, tested, and improved by experienced programmers because AI can sometimes produce incorrect or insecure solutions.

Overall, I believe AI should be used as a supporting tool rather than a replacement for human programmers**.** When used responsibly, AI can improve learning, increase productivity, and help create high-quality software. The best results are achieved when AI is combined with human knowledge, creativity, and critical thinking.

References

- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Google. (2024). *Gemini for Developers*. https://ai.google.dev
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Draw.io. (2024). *Diagrams.net Documentation*. https://www.diagrams.net/
- UNESCO. (2023). *Guidance for Generative AI in Education and Research*. https://unesdoc.unesco.org/
- OpenAI. (2024). *ChatGPT*. https://chatgpt.com
- Anthropic. (2024). *Claude AI*. https://claude.ai/

Git Hub Link

pasinduliyanagamage3-gif/Student-Management-System

……End……